Programming Worlds

PROGRAMMING WORLDS ACADEMY

Git & GitHub 101

From Zero to Your First Pull Request

BEYOND CODE. BEYOND LIMITS.

10

MODULE 10

Undoing Mistakes Safely

Restore files, fix commits, and recover from errors without panic.

MODULE 10 · LESSON 10.1

Undo changes before you commit

التراجع قبل الـ commit

Learning objectives

- Discard a file edit.
- Unstage a file.
- Save work aside with stash.

Prerequisites

Module 4.

Introduction

Most mistakes happen before a commit, and they are the easiest to fix.

Explanation

`git restore index.html` throws away your edits in that file and returns it to the last commit. **This cannot be undone**, because Git never saved those edits.

`git restore --staged style.css` removes a file from the staging area but keeps your edits. It is the opposite of `git add`.

Always run `git status` first so you know which files are staged and which are not.

`git stash push -m "work in progress"` puts your uncommitted edits in a safe drawer and gives you a clean tree. `git stash list` shows drawers and `git stash pop` brings the latest back.

Use stash when you need to switch branches or pull but are not ready to commit.

In VS Code: hover a file in Source Control; the ↶ **Discard Changes** button restores it, and the **–** button unstages. Discard asks for confirmation, and it is also irreversible.

Real-world analogy

> Restore is erasing a pencil line back to the last printed page. Stash is sliding your messy desk into a drawer for later.

Diagram



Figure 1 · Illustration: restore and unstage

Commands

`git restore FILE`
`git restore --staged FILE`
`git stash push -m "msg"`
`git stash pop`

Figure 1 shows restore and unstage.

Common mistakes and solutions

Using restore on work you want to keep.	Stash or commit first.
Confusing restore with restore –staged.	The first discards edits; the second only unstages.
Forgetting stashed work.	Check `git stash list`.

Practical exercise

> Edit index.html, discard the edit. Then stage style.css and unstage it.

Expected result

`git status` shows index.html clean and style.css modified but not staged.

Knowledge check

1. What does git restore FILE do?

- ○ Pushes
- ○ Discards your edits
- ○ Commits

2. What does restore –staged do?

- ○ Unstages but keeps edits
- ○ Deletes the file
- ○ Pushes

3. git stash is for…

- ○ Cloning
- ○ Setting work aside
- ○ Deleting history

Answers

- Discards your edits — It is irreversible.
- Unstages but keeps edits — It undoes git add.
- Setting work aside — Pop brings it back.

Summary

Restore discards edits, restore –staged unstages, and stash parks work.

Key vocabulary

restore	Bring a file back to a saved state
unstage	Remove from staging area
stash	Temporary storage for edits

Next lesson

Lesson 10.2 — Fix a commit.

MODULE 10 · LESSON 10.2

Fix or revert a commit

تعديل أو عكسه commit

Learning objectives

- Amend the last commit.
- Revert a commit.
- Choose between amend and revert.

Prerequisites

Lesson 10.1.

Introduction

Sometimes the mistake is already in a commit. The right tool depends on whether you pushed it.

Explanation

Not pushed yet. `git commit --amend -m "Better message"` replaces the last commit. To add a forgotten file, `git add file` first, then amend.

Amend **rewrites** the commit (it gets a new ID). Never amend a commit others may already have.

Already pushed or shared. `git revert HEAD` creates a **new commit** that cancels the old one. History stays honest and safe for everyone.

To revert an older commit, use its ID from `git log --oneline`: `git revert 8d2e5b1`.

Reverts can conflict; resolve as in Lesson 7.3 and run `git revert --continue`.

After a revert, `git push` as usual.

Real-world analogy

> Amend is fixing a typo in a letter before posting it. Revert is sending a second letter that cancels the first.

Diagram

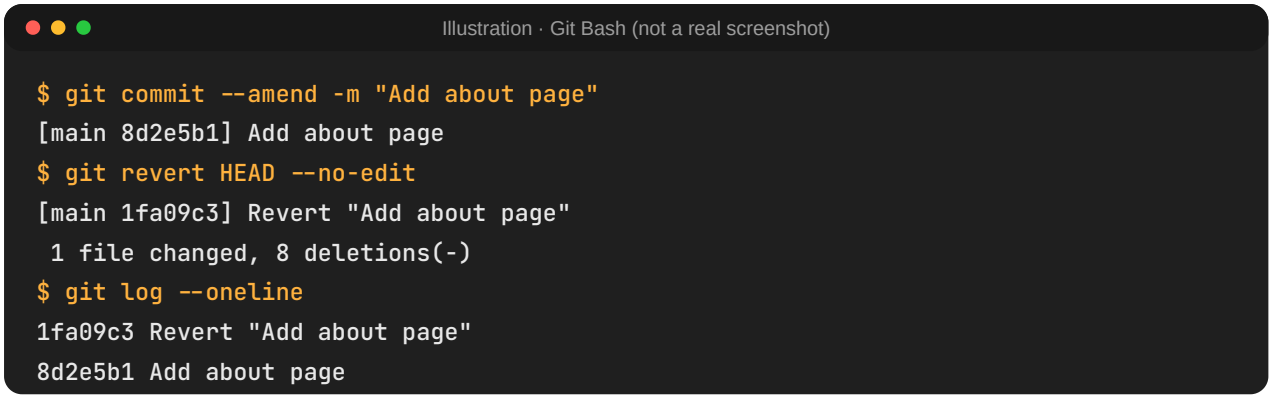


Figure 1 · Illustration: amend and revert

Commands

`git commit --amend -m "msg"`
`git revert HEAD`
`git revert COMMIT-ID`

Figure 1 shows both.

Common mistakes and solutions

Amending after push.	Use revert; amending needs a force push, which is risky.
Reverting the wrong ID.	Check `git log --oneline`.
Expecting revert to delete history.	It adds a new commit.

Practical exercise

> Make a commit with a typo in the message, amend it. Then revert your last commit.

Expected result

`git log --oneline` shows the revert commit on top.

Knowledge check

1. Which is safe for pushed commits?

- ○ git reset –hard
- ○ git revert
- ○ git commit –amend

2. What does amend change?

- ○ The last commit
- ○ All commits
- ○ The remote

3. Does revert delete history?

- ○ Only locally
- ○ No
- ○ Yes

Answers

- git revert — Revert adds a new commit.
- The last commit — It rewrites it.
- No — It adds a cancelling commit.

Summary

Amend before pushing, revert after pushing.

Key vocabulary

amend	Replace the last commit
revert	Commit that cancels another
commit ID	Short hash like 8d2e5b1

Next lesson

Lesson 10.3 — Reset, reflog and common errors.

MODULE 10 · LESSON 10.3

Reset, reflog and common errors

Reset وreflog والأخطاء الشائعة

Learning objectives

- Explain soft, mixed and hard reset.
- Recover with reflog.
- Fix five common errors.

Prerequisites

Lesson 10.2.

Introduction

Reset moves your branch backwards. It is powerful, so learn its three modes.

Explanation

`git reset --soft HEAD~1` undoes the last commit but keeps the changes **staged**.

`git reset HEAD~1` (mixed, the default) keeps the changes in files but **unstaged**.

`git reset --hard HEAD~1` undoes the commit **and discards the changes**. Use it only on local work you do not need.

Never reset commits you already pushed; use revert.

`git reflog` lists where HEAD has been, including commits that look lost. Use `git reset --hard ID` or `git switch -c rescue ID` to get back (Figure 1). Reflog is local and entries expire after some weeks.

Common errors. “Author identity unknown” → set `user.name` and `user.email`. “not a git repository” → `cd` to the project. “nothing to commit” → nothing changed or not staged. “failed to push some refs … fetch first” → `git pull` then push. “detached HEAD” → `git switch main` (or `git switch -c name` to keep the work).

Real-world analogy

> Soft, mixed and hard are three depths of rewinding a tape: keep everything ready, keep the edits, or erase.

Diagram



Figure 1 · Illustration: reset and reflog

Commands

`git reset --soft HEAD~1`
`git reset HEAD~1`
`git reflog`
`git reset --hard ID`

–hard is destructive for uncommitted work.

Common mistakes and solutions

Using –hard casually.	Prefer soft or mixed.
Resetting pushed commits.	Use revert.
Giving up on lost commits.	Look in reflog first.

Practical exercise

> Make a test commit, undo it with reset –soft, then recommit.

Expected result

The file is staged again after reset –soft.

Knowledge check

1. Which reset keeps changes staged?

- ○ –mixed
- ○ –soft
- ○ –hard

2. How do you find a “lost” commit?

- ○ git reflog
- ○ git clone
- ○ git fetch

3. “fetch first” on push means…

- ○ Reinstall
- ○ Pull before pushing
- ○ Delete repo

Answers

- –soft — Soft keeps them staged.
- git reflog — It records HEAD moves.
- Pull before pushing — GitHub has newer commits.

Summary

Reset rewinds locally, reflog rescues, and revert is for shared history.

Key vocabulary

reset	Move the branch to another commit
reflog	Log of HEAD movements
detached HEAD	HEAD on a commit, not a branch

Next lesson

> **Module 11 — Professional habits.**

MODULE 10

Practical lab 10 · Undo safely

1. Edit `index.html` and run `git restore index.html`. Check the edit is gone.
2. Edit `style.css`, run `git add style.css`, then `git restore --staged style.css`. Check with `git status`. Then restore it.
3. Create `test.txt`, commit it as `Add test file`, then run `git commit --amend -m "Add test file for practice"`.
4. Run `git revert HEAD --no-edit`.
5. Create another commit, then `git reset --soft HEAD~1` and recommit it.
6. Run `git reflog` and find your commits.

Check	Pass if…
Restore	Edits discarded as expected
Amend/revert	Log shows the revert commit
Reset/reflog	You recommitted and found it in reflog

Evaluation: 3 of 3 = pass.

Module 10 quiz (pass mark: 70%)

1. Which command discards edits in a file?

- ○ git add FILE
- ○ git push
- ○ git restore FILE

2. Which is safe on pushed commits?

- ○ git commit –amend
- ○ git revert
- ○ git reset –hard

3. reset –soft keeps changes…

- ○ Staged
- ○ Deleted
- ○ Remote

4. How do you find a lost commit?

- ○ git config
- ○ git reflog
- ○ git init

5. Push says “fetch first”. What now?

- ○ Delete the repo
- ○ Force push
- ○ git pull then push

Answers

- git restore FILE — It is irreversible.
- git revert — It adds a new commit.
- Staged — They remain staged.
- git reflog — It tracks HEAD.
- git pull then push — Get the newer commits first.